

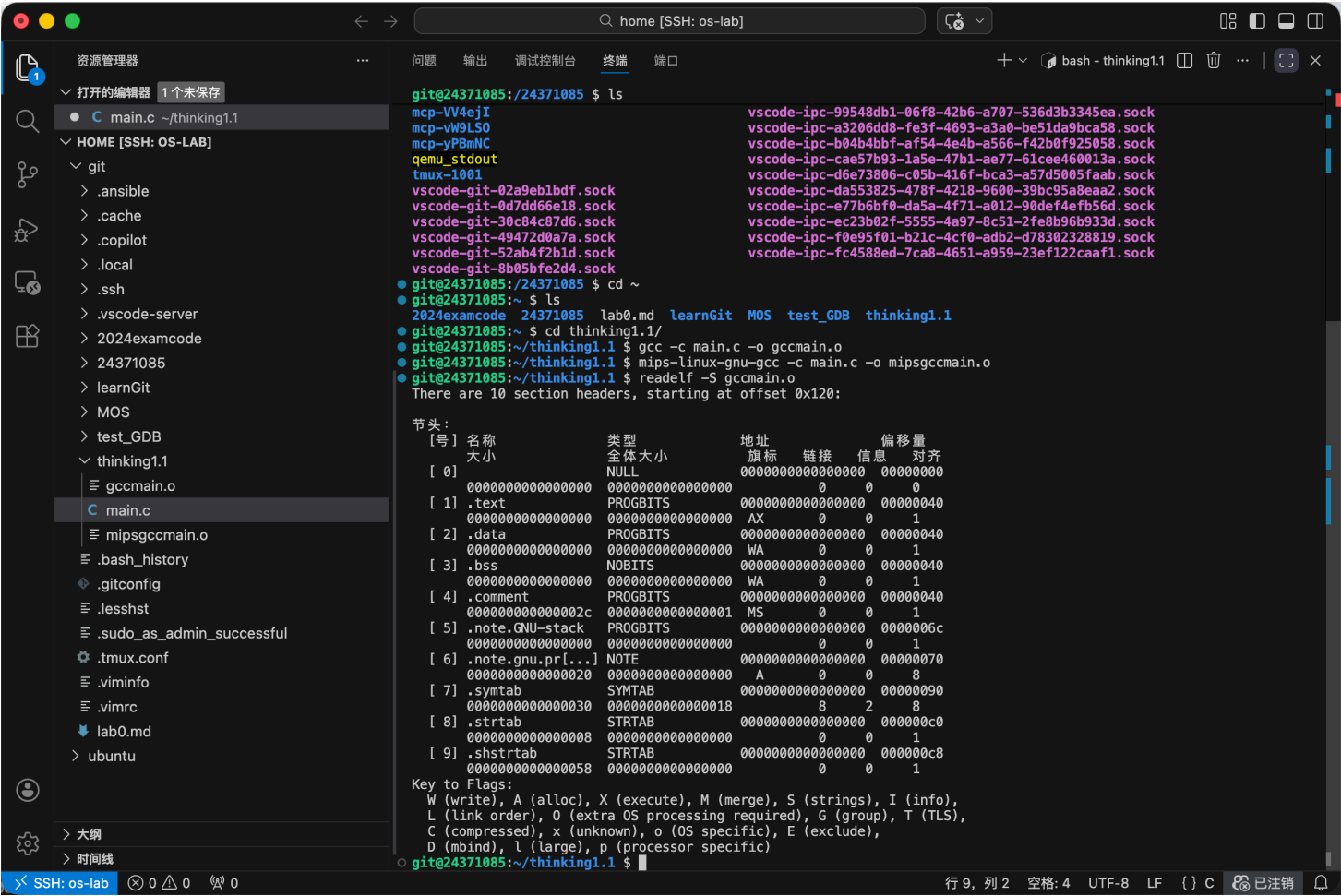
OS-Lab1 实验报告

思考题

Thinking1.1

gcc编译readelf

原生x86



MIPS交叉编译工具链

资源管理器

打开的编辑器 1个未保存

main.c ~/thinking1.1

HOME [SSH: OS-LAB]

git

.ansible

.cache

.copilot

.local

.ssh

.vscode-server

2024examcode

24371085

learnGit

MOS

test_GDB

thinking1.1

gccmain.o

main.c

mipsgccmain.o

.bash_history

.gitconfig

.lessht

.sudo_as_admin_successful

.tmux.conf

.viminfo

.vimrc

lab0.md

ubuntu

大纲

时间线

问题 输出 调试控制台 终端 端口

bash - thinking1.1

git@24371085:~/thinking1.1 \$ readelf -S gccmain.o

0000000000000000 0000000000000000 0 0 0

[1] .text PROGBITS 0000000000000000 0000000000000000 00000040

0000000000000000 0000000000000000 AX 0 0 1

[2] .data PROGBITS 0000000000000000 0000000000000000 00000040

0000000000000000 0000000000000000 WA 0 0 1

[3] .bss NOBITS 0000000000000000 0000000000000000 00000040

0000000000000000 0000000000000000 WA 0 0 1

[4] .comment PROGBITS 0000000000000000 0000000000000000 00000040

000000000000002c 0000000000000001 MS 0 0 1

[5] .note.GNU-stack PROGBITS 0000000000000000 0000000000000000 0000006c

0000000000000000 0000000000000000 0 0 1

[6] .note.gnu.pr[...] NOTE 0000000000000000 0000000000000000 00000070

0000000000000020 0000000000000000 A 0 0 8

[7] .symtab SYMTAB 0000000000000000 0000000000000000 00000090

0000000000000030 0000000000000018 8 2 8

[8] .strtab STRTAB 0000000000000000 0000000000000000 000000c0

0000000000000008 0000000000000000 0 0 1

[9] .shstrtab STRTAB 0000000000000000 0000000000000000 000000c8

0000000000000058 0000000000000000 0 0 1

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),

L (link order), O (extra OS processing required), G (group), T (TLS),

C (compressed), x (unknown), o (OS specific), E (exclude),

D (mbind), l (large), p (processor specific)

git@24371085:~/thinking1.1 \$ readelf -S mipsgccmain.o

There are 14 section headers, starting at offset 0x1f0:

节头:

[Nr]	Name	Type	Addr	Off	Size	ES	Flg	Lk	Inf	Al
[0]		NULL	00000000	000000	000000	00		0	0	0
[1]	.text	PROGBITS	00000000	000040	000000	00	AX	0	0	16
[2]	.data	PROGBITS	00000000	000040	000000	00	WA	0	0	16
[3]	.bss	NOBITS	00000000	000040	000000	00	WA	0	0	16
[4]	.reginfo	MIPS_REGINFO	00000000	000040	000018	18	A	0	0	4
[5]	.MIPS.abiflags	MIPS_ABIFLAGS	00000000	000058	000018	18	A	0	0	8
[6]	.pdr	PROGBITS	00000000	000070	000000	00		0	0	4
[7]	.mdebug.abi32	PROGBITS	00000000	000070	000000	00		0	0	1
[8]	.comment	PROGBITS	00000000	000070	000027	01	MS	0	0	1
[9]	.note.GNU-stack	PROGBITS	00000000	000097	000000	00		0	0	1
[10]	.gnu.attributes	GNU_ATTRIBUTES	00000000	000097	000010	00		0	0	1
[11]	.symtab	SYMTAB	00000000	0000a8	0000c0	10		12	12	4
[12]	.strtab	STRTAB	00000000	000168	000008	00		0	0	1
[13]	.shstrtab	STRTAB	00000000	000170	000008	00		0	0	1

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),

L (link order), O (extra OS processing required), G (group), T (TLS),

C (compressed), x (unknown), o (OS specific), E (exclude),

D (mbind), p (processor specific)

git@24371085:~/thinking1.1 \$

行 9, 列 2 空格: 4 UTF-8 LF {} C 已注销

```
git@24371085:~/thinking1.1 $ readelf -S gccmain.o
git@24371085:~/thinking1.1 $ readelf -S mipsgccmain.o
There are 14 section headers, starting at offset 0x1f0:

节头:
[Nr] Name           Type             Addr      Off      Size    ES Flg Lk Inf Al
[ 0]                NULL             00000000  000000  000000  00          0  0  0
[ 1] .text             PROGBITS         00000000  000040  000000  00  AX  0  0 16
[ 2] .data             PROGBITS         00000000  000040  000000  00  WA  0  0 16
[ 3] .bss              NOBITS           00000000  000040  000000  00  WA  0  0 16
[ 4] .reginfo          MIPS_REGINFO     00000000  000040  000018  18  A   0  0  4
[ 5] .MIPS.abiflags    MIPS_ABIFLAGS    00000000  000058  000018  18  A   0  0  8
[ 6] .pdr              PROGBITS         00000000  000070  000000  00          0  0  4
[ 7] .mdebug.abi32     PROGBITS         00000000  000070  000000  00          0  0  1
[ 8] .comment          PROGBITS         00000000  000070  000027  01  MS   0  0  1
[ 9] .note.GNU-stack   PROGBITS         00000000  000097  000000  00          0  0  1
[10] .gnu.attributes   GNU_ATTRIBUTES   00000000  000097  000010  00          0  0  1
[11] .symtab           SYMTAB           00000000  0000a8  0000c0  10  12 12  4
[12] .strtab           STRTAB           00000000  000168  000008  00          0  0  1
[13] .shstrtab         STRTAB           00000000  000170  000080  00          0  0  1

Key to Flags:
W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
D (mbind), p (processor specific)

git@24371085:~/thinking1.1 $ mips-linux-gnu-readelf -S mipsgccmain.o
There are 14 section headers, starting at offset 0x1f0:

节头:
[Nr] Name           Type             Addr      Off      Size    ES Flg Lk Inf Al
[ 0]                NULL             00000000  000000  000000  00          0  0  0
[ 1] .text             PROGBITS         00000000  000040  000000  00  AX  0  0 16
[ 2] .data             PROGBITS         00000000  000040  000000  00  WA  0  0 16
[ 3] .bss              NOBITS           00000000  000040  000000  00  WA  0  0 16
[ 4] .reginfo          MIPS_REGINFO     00000000  000040  000018  18  A   0  0  4
[ 5] .MIPS.abiflags    MIPS_ABIFLAGS    00000000  000058  000018  18  A   0  0  8
[ 6] .pdr              PROGBITS         00000000  000070  000000  00          0  0  4
[ 7] .mdebug.abi32     PROGBITS         00000000  000070  000000  00          0  0  1
[ 8] .comment          PROGBITS         00000000  000070  000027  01  MS   0  0  1
[ 9] .note.GNU-stack   PROGBITS         00000000  000097  000000  00          0  0  1
[10] .gnu.attributes   GNU_ATTRIBUTES   00000000  000097  000010  00          0  0  1
[11] .symtab           SYMTAB           00000000  0000a8  0000c0  10  12 12  4
[12] .strtab           STRTAB           00000000  000168  000008  00          0  0  1
[13] .shstrtab         STRTAB           00000000  000170  000080  00          0  0  1

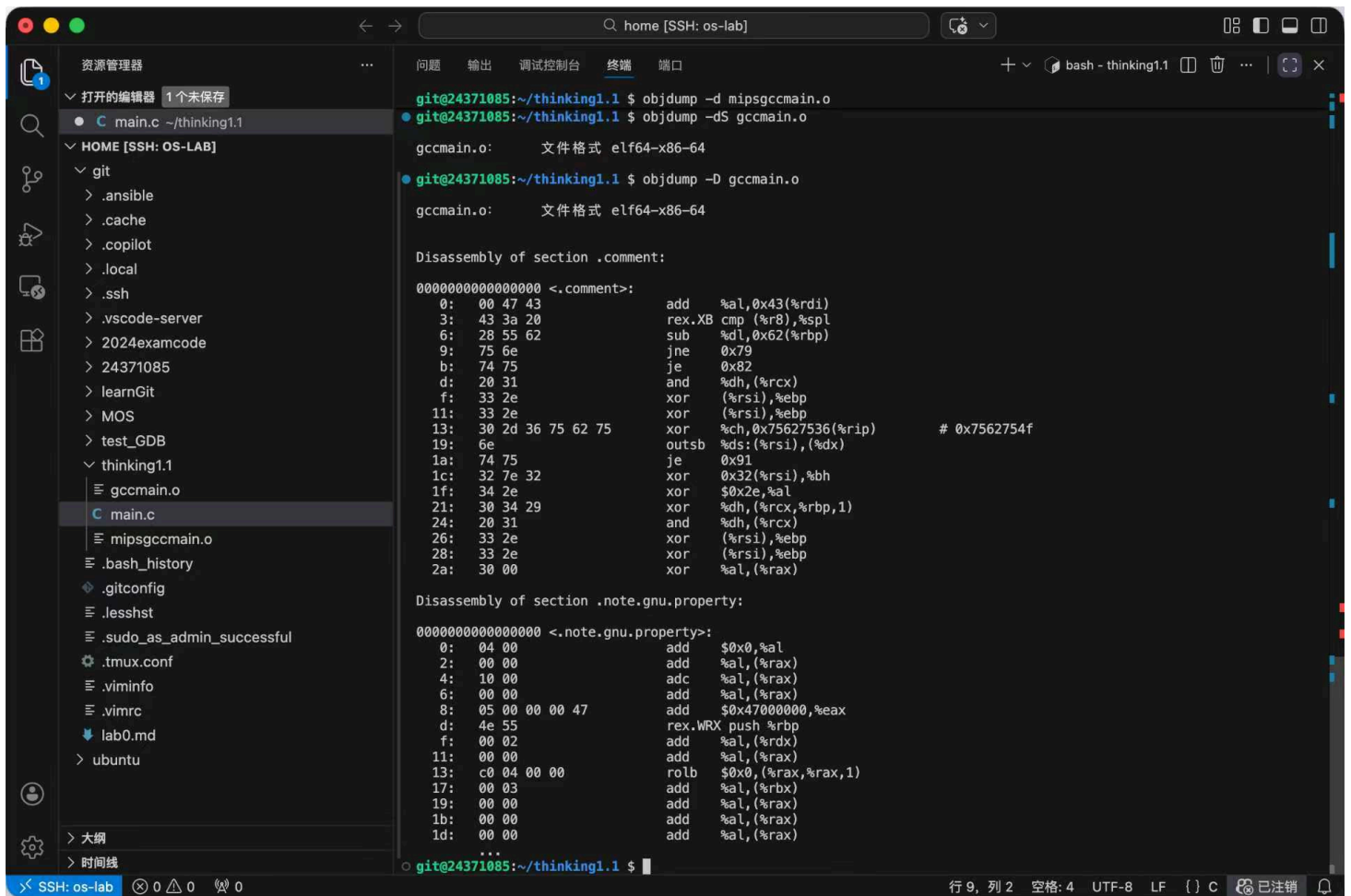
Key to Flags:
W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
D (mbind), p (processor specific)

git@24371085:~/thinking1.1 $
```

readelf在不使用mips交叉编译工具链的时候也可以正确阅读由mips交叉编译工具链编译得到的内容。

objdump

用原生x86的objdump反编译gccmain.o结果如下：



```
git@24371085:~/thinking1.1 $ objdump -d mipsgccmain.o
git@24371085:~/thinking1.1 $ objdump -ds gccmain.o

gccmain.o:          文件格式 elf64-x86-64

git@24371085:~/thinking1.1 $ objdump -D gccmain.o

gccmain.o:          文件格式 elf64-x86-64

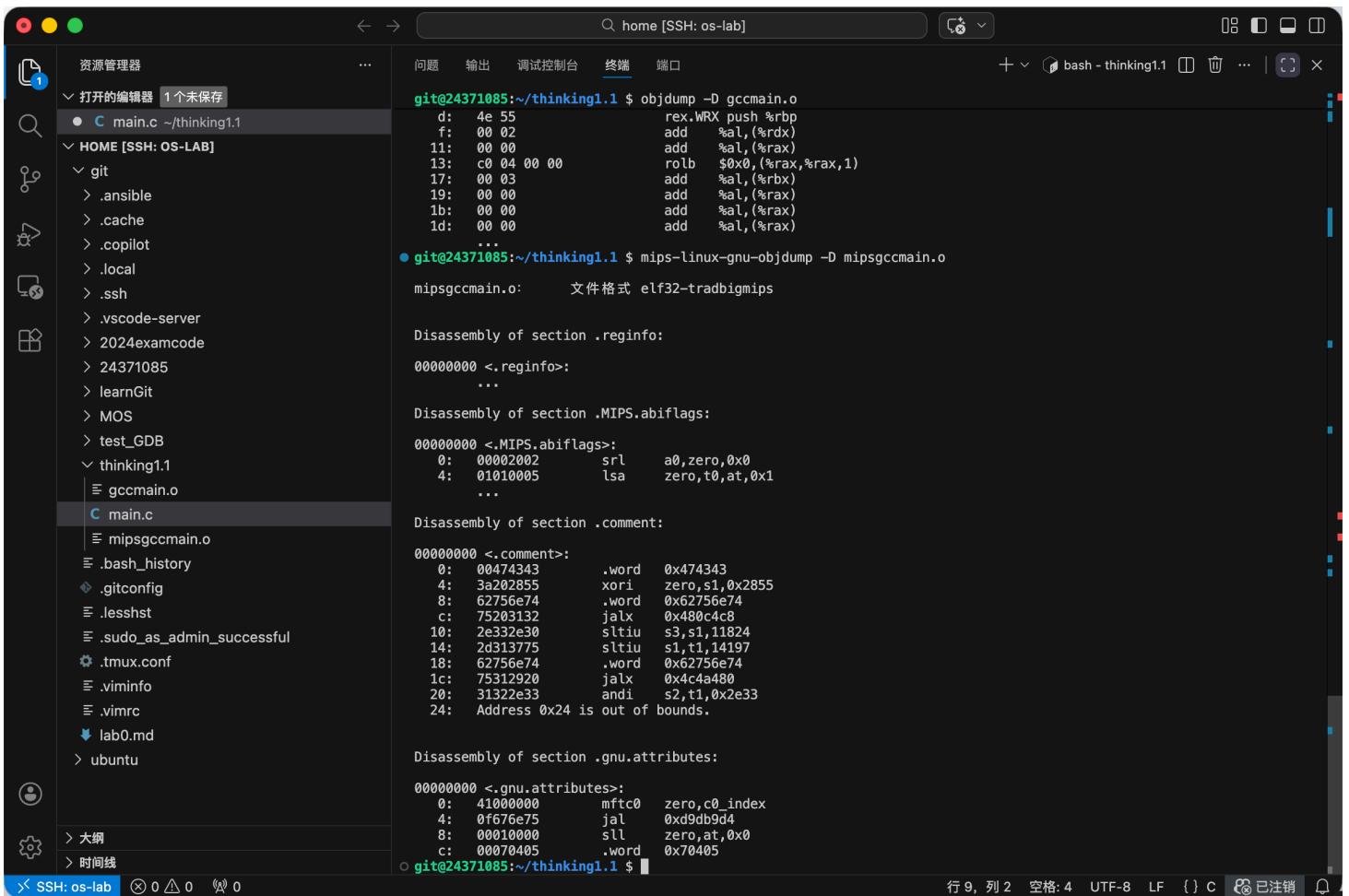
Disassembly of section .comment:

0000000000000000 <.comment>:
 0: 00 47 43          add    %al,0x43(%rdi)
 3: 43 3a 20          rex.XB cmp    (%r8),%spl
 6: 28 55 62          sub    %dl,0x62(%rbp)
 9: 75 6e            jne    0x79
 b: 74 75            je     0x82
 d: 20 31          and    %dh,(%rcx)
 f: 33 2e          xor    (%rsi),%ebp
11: 33 2e          xor    (%rsi),%ebp
13: 30 2d 36 75 62 75  xor    %ch,0x75627536(%rip)    # 0x7562754f
19: 6e            outsb  %ds:(%rsi),(%dx)
1a: 74 75            je     0x91
1c: 32 7e 32          xor    0x32(%rsi),%bh
1f: 34 2e          xor    $0x2e,%al
21: 30 34 29          xor    %dh,(%rcx,%rbp,1)
24: 20 31          and    %dh,(%rcx)
26: 33 2e          xor    (%rsi),%ebp
28: 33 2e          xor    (%rsi),%ebp
2a: 30 00          xor    %al,(%rax)

Disassembly of section .note.gnu.property:

0000000000000000 <.note.gnu.property>:
 0: 04 00          add    $0x0,%al
 2: 00 00          add    %al,(%rax)
 4: 10 00          adc    %al,(%rax)
 6: 00 00          add    %al,(%rax)
 8: 05 00 00 00 47    add    $0x47000000,%eax
 d: 4e 55          rex.WRX push %rbp
 f: 00 02          add    %al,(%rdx)
11: 00 00          add    %al,(%rax)
13: c0 04 00 00      rolb   $0x0,(%rax,%rax,1)
17: 00 03          add    %al,(%rbx)
19: 00 00          add    %al,(%rax)
1b: 00 00          add    %al,(%rax)
1d: 00 00          add    %al,(%rax)
...
```

用MIPS交叉编译工具链的objdump反编译mipsgccmain.o结果如下：



```
git@24371085:~/thinking1.1 $ objdump -D gccmain.o
d: 4e 55 rex.WRX push %rbp
f: 00 02 add %al, (%rdx)
11: 00 00 add %al, (%rax)
13: c0 04 00 00 rolb $0x0, (%rax,%rax,1)
17: 00 03 add %al, (%rbx)
19: 00 00 add %al, (%rax)
1b: 00 00 add %al, (%rax)
1d: 00 00 add %al, (%rax)
...
git@24371085:~/thinking1.1 $ mips-linux-gnu-objdump -D mipsgccmain.o

mipsgccmain.o: 文件格式 elf32-tradbigmips

Disassembly of section .reginfo:

00000000 <.reginfo>:
...

Disassembly of section .MIPS.abiflags:

00000000 <.MIPS.abiflags>:
0: 00002002 srl a0,zero,0x0
4: 01010005 lsa zero,t0,at,0x1
...

Disassembly of section .comment:

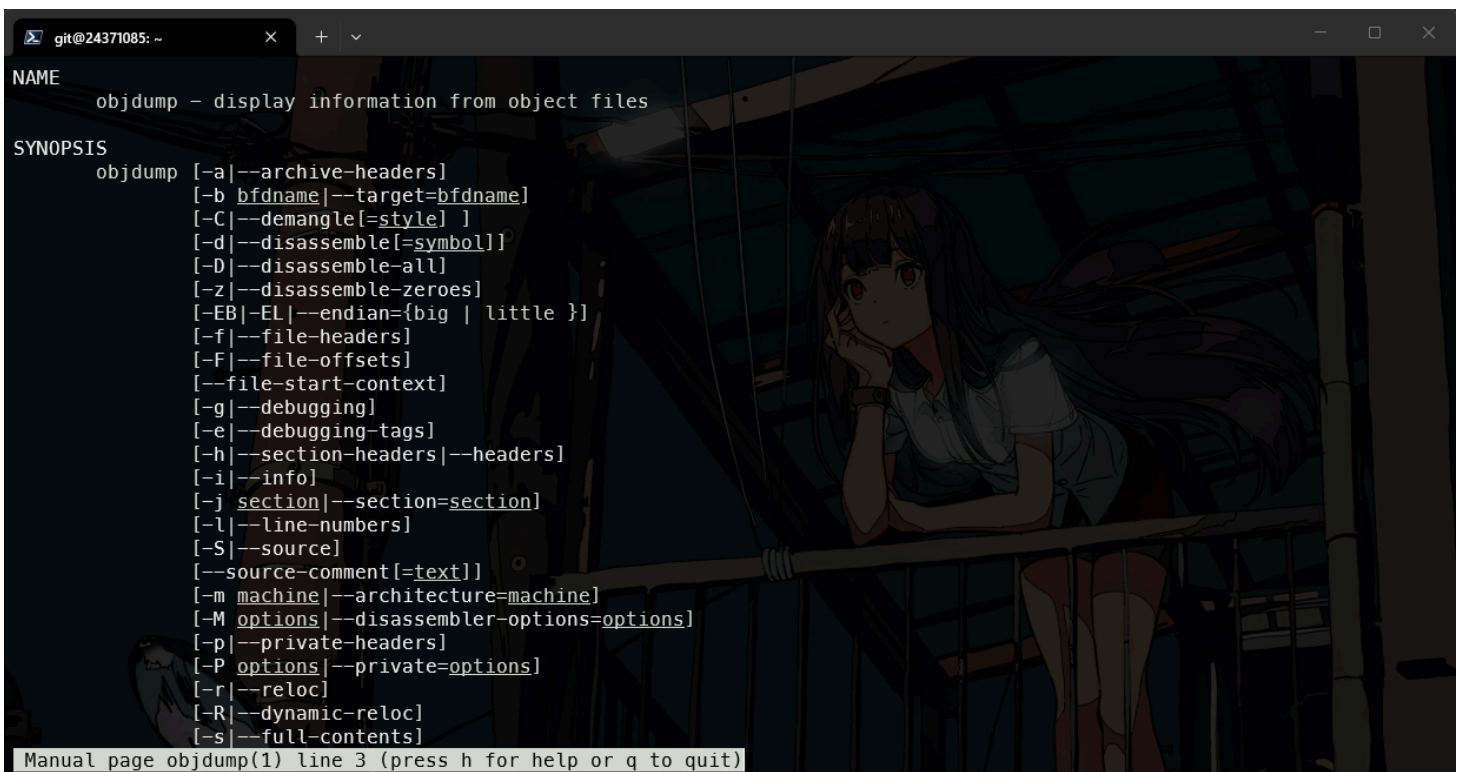
00000000 <.comment>:
0: 00474343 .word 0x474343
4: 3a202855 xori zero,s1,0x2855
8: 62756e74 .word 0x62756e74
c: 75203132 jalx 0x480c4c8
10: 2e332e30 sltiu s3,s1,11824
14: 2d313775 sltiu s1,t1,14197
18: 62756e74 .word 0x62756e74
1c: 75312920 jalx 0x4c4a480
20: 31322e33 andi s2,t1,0x2e33
24: Address 0x24 is out of bounds.

Disassembly of section .gnu.attributes:

00000000 <.gnu.attributes>:
0: 41000000 mftc0 zero,c0_index
4: 0f676e75 jal 0xd9db9d4
8: 00010000 sll zero,at,0x0
c: 00070405 .word 0x70405
git@24371085:~/thinking1.1 $
```

objdump的参数含义

man objdump



```
NAME
    objdump - display information from object files

SYNOPSIS
    objdump [-a|--archive-headers]
             [-b bfdname|--target=bfdname]
             [-C|--demangle=[style] ]
             [-d|--disassemble=[symbol]]
             [-D|--disassemble-all]
             [-Z|--disassemble-zeroes]
             [-EB|-EL|--endian={big | little }]
             [-f|--file-headers]
             [-F|--file-offsets]
             [--file-start-context]
             [-g|--debugging]
             [-e|--debugging-tags]
             [-h|--section-headers|--headers]
             [-i|--info]
             [-j section|--section=section]
             [-l|--line-numbers]
             [-S|--source]
             [--source-comment=[text]]
             [-m machine|--architecture=machine]
             [-M options|--disassembler-options=options]
             [-p|--private-headers]
             [-P options|--private=options]
             [-r|--reloc]
             [-R|--dynamic-reloc]
             [-s|--full-contents]

Manual page objdump(1) line 3 (press h for help or q to quit)
```

其中比较常用的如下：

- e：显示调试信息
- D：反汇编所有section
- r：显示文件的重定位入口
- S：尽可能反汇编出源代码
- z：将0也进行反汇编

Thinking1.2

用readelf解析mos

```
git@24371085:~/24371085/tools/readelf (lab1-pre-exam)$ ./readelf ../../target/mos
0:0x0
1:0x80020000
2:0x80020f10
3:0x80020f28
4:0x80020f40
5:0x800210c0
6:0x0
7:0x0
8:0x0
9:0x0
10:0x0
11:0x0
```

用readelf解析文件本身

无输出结果。

可以用 `readelf -h` 来研究文件头

用readelf分析mos与readelf.o

其类别分别为ELF32和ELF64。~~~~

在readelf.c中，我们采用的都是ELF32类型的数据结构，因此不能用来解析ELF64文件。

Thinking1.3

- 为什么操作系统的内核入口并没有放在上电启动地址，还能保证内核入口被正确跳转到？
 - 上电启动位置是用于启动bootloader，实现硬件初始化等内容。而bootloader所加载的内容才决定了内核的入口，也就是start.S中写的_start函数才是mips_init的入口。

难点分析

readelf文件的实现

.elf包含.exe,.o和.so，即可执行文件，可重定位文件和共享对象文件。可以用readelf解析hello，readelf，readelf.o，main.o等多种文件。

在elf.h中，对ELF32的种种内容做出了定义。即 Elf32_Ehdr`Elf32\_Shdr`Elf32_phdr 三种类型，分别描述ELF头，节头表和段头表。

实现的readelf是只能解析ELF32的section地址的可执行程序。使用shdr指针定位各个section，从而输出地址。让sh_table指针指向节头表第一项的地址：`binary + ehdr->e_shoff`，接下来循环输出即可。

注意需将sh_table这一指针变为Elf_Shdr*类型。

printk函数的实现

vprintfmt的参数：out函数指针，data指针，fmt字符串，ap数组。

out函数指针能实现在同一函数中调用不同形式的out函数，可以实现输出以buf为起点，长度为len的字符串。

我们需要补充的核心逻辑就是将 `%x` 替换为对应的数据。即找到%并解析%后面的内容。每当找到%时就讲前面内容全部out输出，找到%时通过对其后内容解析来改变参数即可。

实验体会

Lab1不同于Lab0的前置知识，一下需要阅读大量的代码，初期的实验推进十分困难。指导书的阅读也频频受阻。（为了降低不必要的负担，我还是选择使用vscode进行代码阅读）

为了接下来的Lab顺利进行，完整的项目阅读是很有必要的，虽然有许多暂时与Lab1无关，但是多花些时间完整梳理整个脉络想必对以后大有益处。